

SAPIENZA
UNIVERSITÀ DI ROMA

The Locked Archive: Escape Room Solver

Planning & Reasoning

A.Y. 2025/2026

Bogdan Andrei Tutuianu
2217841

TABLE OF CONTENTS

Part 1 — PDDL
• Domain formulation (rooms, locks, items, combine puzzle)
• Three problem instances of increasing complexity
• Search algorithm & heuristic comparison

Part 2 — Situation Calculus & IndiGolog
• Action theory (fluents, poss, successor state axioms)
• Reasoning tasks: Legality, Projection, Regression
• Simple & Reactive controllers with exogenous events

The Locked Archive

An agent wakes up trapped inside a multi-room archive.
To escape, the agent must:

- Explore connected rooms
- Collect items scattered across the archive
- Read clues to decode numeric codes
- Combine two carried items to craft a new one, when needed
- Unlock doors — with a key, or with a known code
- Reach the designated exit room

Two kinds of locked doors:

🔑 Key locks — opened by carrying the matching item
🔢 Code locks — opened once the code has been learned from a clue

PDDL Domain: Types & Predicates

Types
room, item, lock - object key_lock, code_lock - lock

Functions (move-cost ?l - lock) (total-cost)

Predicates
(at ?r - room) agent's current room
(connected ?r1 ?r2 - room ?l - lock) static room adjacency
(unlocked ?l - lock) lock state (item_at ?i - item ?r - room) item location
 (has ?i - item) agent possession
 (needs_key ?l - key_lock ?i - item) static key requirement
 (known_code ?l - code_lock) code learned?
 (combinable ?i1 ?i2 ?i3 - item) static combine recipe
(clue_at ?l - code_lock ?r - room) static clue location

PDDL Actions — move & pick_up

```
(:action move
  :parameters (?from ?to - room ?l - lock)
  :precondition (and
    (at ?from)
    (connected ?from ?to ?l)
    (unlocked ?l)
  )
  :effect (and
    (not (at ?from))
    (at ?to)
    (increase (total-cost) (move-cost ?l))
  )
)
```

```
(:action pick_up
 :parameters (?i - item ?r - room)
 :precondition (and
 (at ?r)
 (item_at ?i ?r)
 )
 :effect (and
 (not (item_at ?i ?r))
 (has ?i)
 )
)
```

PDDL Actions:
read_clue, unlock_with_key, unlock_with_code

(:action read_clue :parameters (?l - code_lock ?r - room)
:precondition (and (at ?r) (clue_at ?l ?r)) :effect
(known_code ?l)

(:action unlock_with_code
:parameters (?l - code_lock)
:precondition (known_code ?l) :effect
(unlocked ?l))

(:action unlock_with_key :parameters (?l - key_lock ?i
- item) :precondition (and (needs_key ?l ?i) (has ?i))
:effect (unlocked ?l))

PDDL ACTIONS — combine adl

```
(:action combine
  :parameters (?i1 ?i2 - item)
  :precondition (and
    (has ?i1)
    (has ?i2)
    (exists (?i3 - item) (combinable ?i1 ?i2 ?i3))
  )
  :effect (and
    (not (has ?i1))
    (not (has ?i2))
    (forall (?i3 - item)
      (when (combinable ?i1 ?i2 ?i3)
        (has ?i3)
      )
    )
  )
)
```

PDDL ACTIONS — combine strips

```
(:action combine
  :parameters (?i1 ?i2 ?i3 - item)
  :precondition (and
    (has ?i1)
    (has ?i2)
    (combinable ?i1 ?i2 ?i3)
  )
  :effect (and
    (not (has ?i1))
    (not (has ?i2))
    (has ?i3)
  )
)
```

PDDL Problems instaces

Easy (3 rooms)
• 1 key lock, 1 code lock • No combine step — sanity check

Medium (5 rooms)
• 2 key locks, 2 code locks • 1 mandatory combine (item_x + item_y → key_c)

Hard (8 rooms, two branches)
• West branch: mandatory combine, cheap doors (move-cost 3) • East branch: decoy combine, expensive doors (move-cost 5) • Cost-optimal plan = west branch

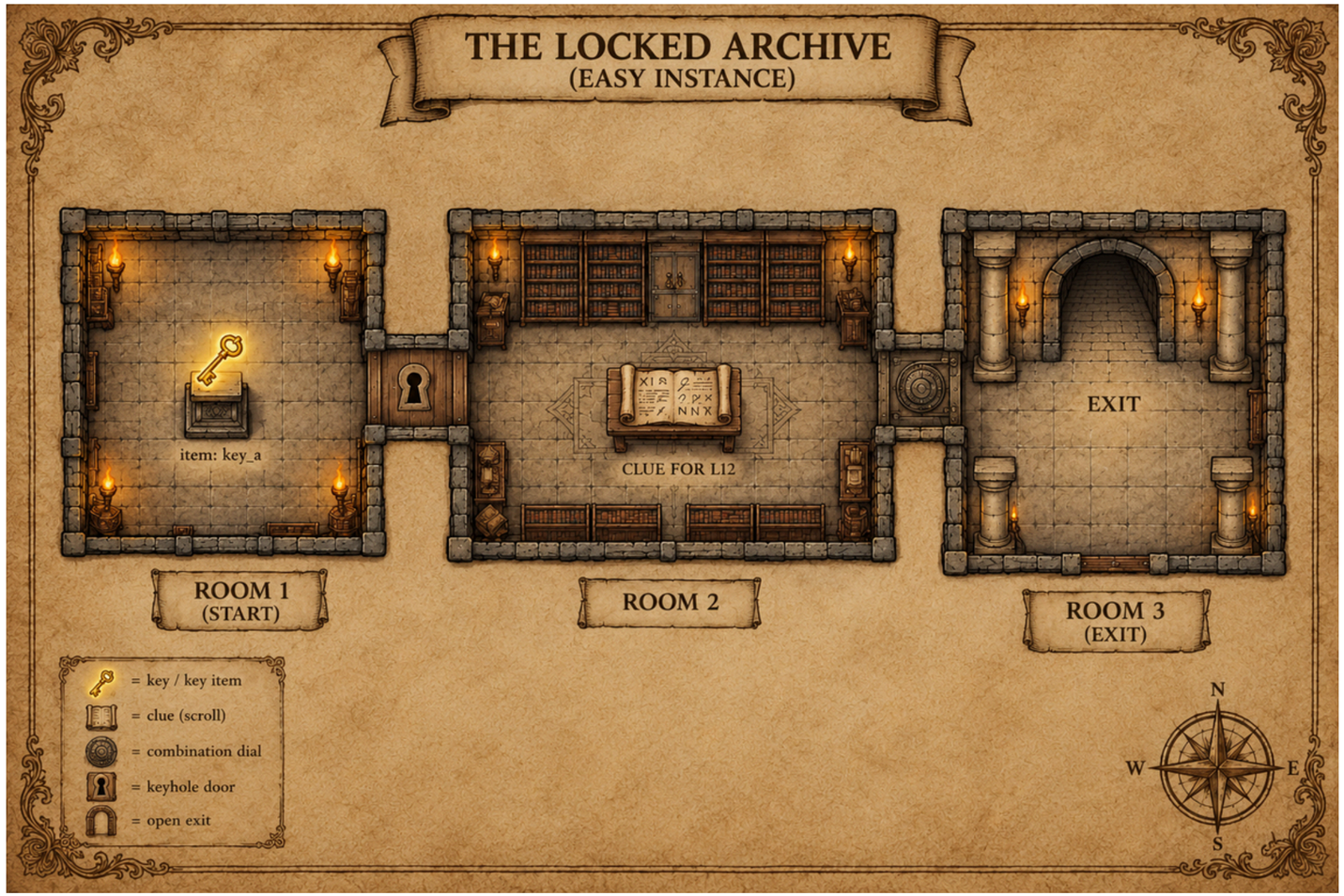
Easy instance

```
 (:objects
 r1 r2 r3 - room
 l1 - key_lock
 l2 - code_lock
 key1 - item )

(:init
(at r1)
 (connected r1 r2 l1)
 (connected r2 r1 l1)
(connected r2 r3 l2)
 (connected r3 r2 l2)
(needs_key l1 key1)
 (item_at key1 r1)
 (clue_at l2 r2)
 (= (move-cost l1) 1)
 (= (move-cost l2) 1)
 (= (total-cost) 0) )

(:goal (at r3))

 (:metric minimize (total-cost)) )
```



Easy Instance Performance

All three configurations reach the same optimal plan (cost 2, 6 actions).
Blind search expands slightly more states (9 vs 7) than the heuristic-guided searches, since it has no information to prioritize promising states — but on an instance this small, the difference is negligible.

Search	Heuristic	Domain	Plan Length	Plan Cost	Expanded States	Search Time
A*	Blind	ADL	6	2	9	0.14 ms
A*	hFF	STRIPS	6	2	7	0.23 ms
A*	hadd	STRIPS	6	2	7	0.27 ms

Medium instance

(:objects
 r1 r2 r3 r4 r5 - room
l1 l3 - key_lock
 l2 l4 - code_lock
 key_a item_x item_y key_c - item)

 (:init
 (at r1)
(connected r1 r2 l1)
(connected r2 r1 l1)
(connected r2 r3 l2)
 (connected r3 r2 l2)
(connected r3 r4 l3)
 (connected r4 r3 l3)
 (connected r4 r5 l4)
 (connected r5 r4 l4)
 (needs_key l1 key_a)
(needs_key l3 key_c)
 (item_at key_a r1)
(item_at item_x r2)
 (item_at item_y r3)
 (combinable item_x item_y key_c)
 (clue_at l2 r1) (clue_at l4 r4)
 (= (move-cost l1) 1)
 (= (move-cost l2) 1)
 (= (move-cost l3) 1)
(= (move-cost l4) 1)
 (= (total-cost) 0))



(:goal (at r5))
(:metric minimize (total-cost)))

Medium Instance Performance

Blind and hFF find the optimal plan (cost 4). hadd instead returns a cost-6 plan — it takes an unnecessary round trip (move(r1,r2) → move(r2,r1) → read_clue(l2,r1) → move(r1,r2) again) instead of reading the clue before ever leaving r1. This is our first concrete case of heuristic inadmissibility.

Search	Heuristic	Domain	Plan Length	Plan Cost	Expanded States	Search Time
A*	Blind	ADL	14	4	38	0.22 ms
A*	hFF	STRIPS	14	4	19	0.37 ms
A*	hadd	STRIPS	16	6	18	0.29 ms

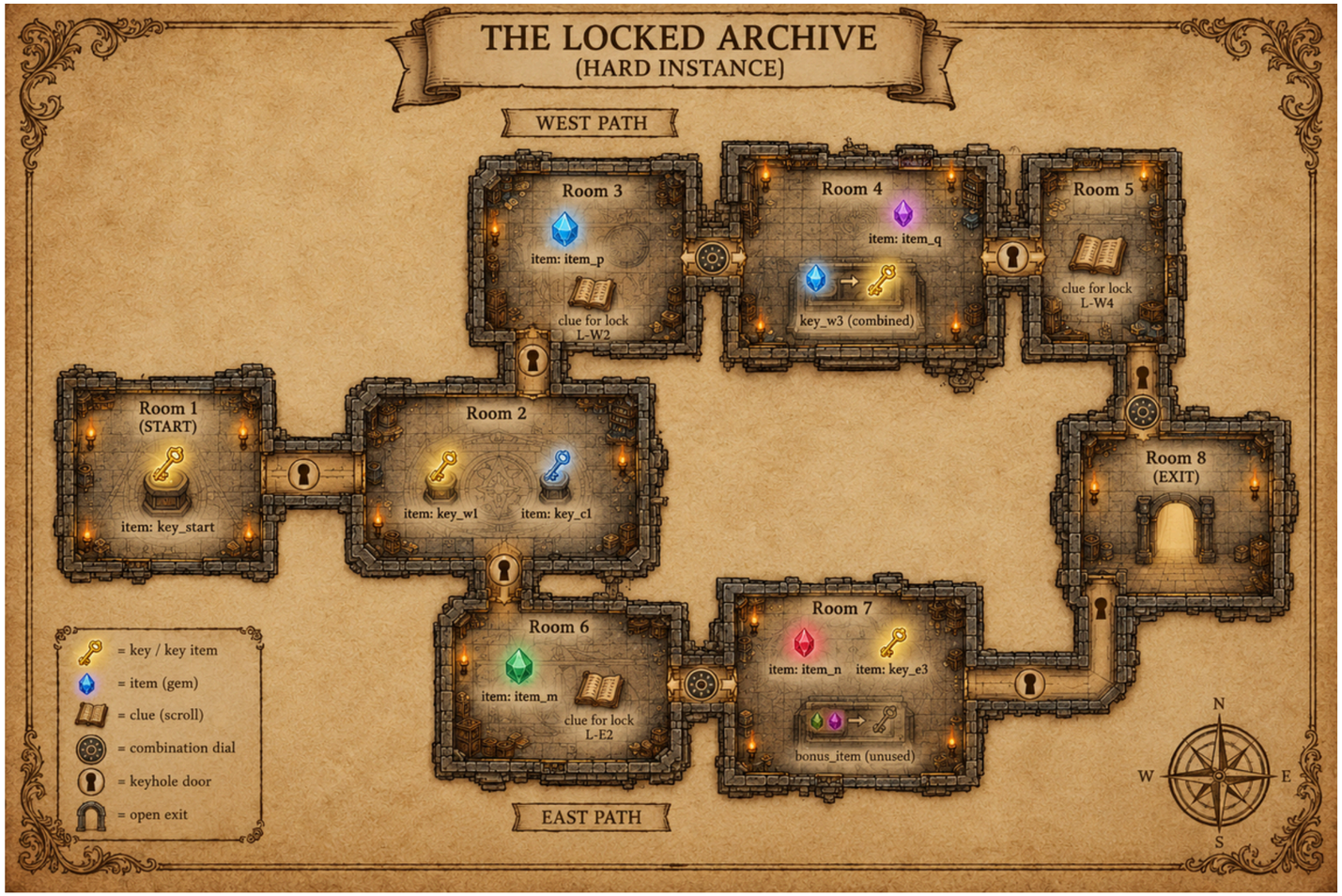
hard instance

(:objects
r1 r2 r3 r4 r5 r6 r7 r8 - room
l0 l_w1 l_w3 l_e1 l_e3 - key_lock
l_w2 l_w4 l_e2 - code_lock
key_start key_w1 item_p item_q key_w3 key_e1
item_m item_n key_e3 bonus_item - item)

(:init
(at r1)
(connected r1 r2 l0) (connected r2 r1 l0)
;; west branch (cheap, mandatory combine)
(connected r2 r3 l_w1) (connected r3 r2 l_w1)
(connected r3 r4 l_w2) (connected r4 r3 l_w2)
(connected r4 r5 l_w3) (connected r5 r4 l_w3)
(connected r5 r8 l_w4) (connected r8 r5 l_w4)
;; east branch (expensive, decoy combine)
(connected r2 r6 l_e1) (connected r6 r2 l_e1)
(connected r6 r7 l_e2) (connected r7 r6 l_e2)
(connected r7 r8 l_e3) (connected r8 r7 l_e3)
(needs_key l0 key_start)
(needs_key l_w1 key_w1) (needs_key l_w3 key_w3)
(needs_key l_e1 key_e1) (needs_key l_e3 key_e3)
(item_at key_start r1) (item_at key_w1 r2)
(item_at item_p r3) (item_at item_q r4)
(item_at key_e1 r2) (item_at item_m r6)
(item_at item_n r7) (item_at key_e3 r7)
(combinable item_p item_q key_w3) ;; mandatory, west
(combinable item_m item_n bonus_item) ;; decoy,east
(clue_at l_w2 r3) (clue_at l_w4 r5) (clue_at l_e2 r6)
(= (move-cost l0) 1) (= (move-cost l_w1) 3) (= (move-cost l_w2) 3) (=
(move-cost l_w3) 3) (= (move-cost l_w4) 3) (= (move-cost l_e1) 5) (=
(move-cost l_e2) 5) (= (move-cost l_e3) 5) (= (total-cost) 0))
l

(:goal (at r8)) (:metric minimize (total-cost))

hard Instance



hard Instance Performance

Blind search — exhaustive and admissible — correctly finds the cost-optimal west branch (cost 13, 17 actions, mandatory combine), at the price of expanding 169 states. Both hFF and hadd instead settle for the east branch (cost 16, only 12 actions) — fewer actions, cheaper to search (~11× fewer expanded states), but not optimal.

Search	Heuristic	Domain	Plan Length	Plan Cost	Expanded States	Search Time
A*	Blind	ADL	17	13	169	0.61 ms
A*	hFF	STRIPS	12	16	15	0.31 ms
A*	hadd	STRIPS	12	16	15	0.25 ms

pick_up key_start r1
unlock_with_key l0 key_start
move r1 r2 l0
pick_up key_w1 r2
unlock_with_key l_w1 key_w1
move r2 r3 l_w1
pick_up item_p r3
read_clue l_w2 r3
unlock_with_code l_w2
move r3 r4 l_w2
pick_up item_q r4
combine item_p item_q
unlock_with_key l_w3 key_w3
move r4 r5 l_w3
read_clue l_w4 r5
unlock_with_code l_w4
move r5 r8 l_w4

pick_up key_start r1
unlock_with_key l0 key_start
move r1 r2 l0
pick_up key_e1 r2
unlock_with_key l_e1 key_e1
move r2 r6 l_e1
read_clue l_e2 r6
unlock_with_code l_e2
move r6 r7 l_e2
pick_up key_e3 r7
unlock_with_key l_e3 key_e3
move r7 r8 l_e3

finding

HEURISTIC INADMISSIBILITY

Both hFF and hadd are based on delete relaxation — they ignore delete effects when estimating cost-to-go, which can make an expensive path look artificially cheap.

Medium instance: hadd returns cost 6 instead of the optimal 4
Hard instance: hFF and hadd both return cost 16 instead of the optimal 13

Trade-off demonstrated:

Blind Search	hFF / hadd
Admissible	Inadmissible
Always optimal	Can miss the optimum
Expands far more states (169 on Hard)	~10× fewer expanded states (15 on Hard)

This is not a bug — it's the expected, textbook trade-off between search effort and solution optimality, made concrete on a domain specifically designed to hide a cheap optimum behind a puzzle.

finding

DEVIATION FROM THE PROPOSAL

Original proposal:
time_remaining (numeric fluent) + time_cost(action)
→ decreasing numeric precondition

Why dropped:
Fast Downward does not support a decreasing numeric precondition of this shape without PDDL extensions outside the taught subset — would have blocked heuristic search entirely.

Replacement:
Standard :action-costs pattern
→ single total-cost function
→ move-cost differs per lock/door
(west doors: 3 · east doors: 5)

Same design goal achieved: forces the planner to discard an inefficient route under a resource constraint — with a formulation Fast Downward and its heuristics fully support.

Indigolo overview

PART 2 — SITUATION CALCULUS & INDIGOLOG

Same domain, different formalism:

- High-level agent programming instead of classical planning
- Online execution, not just offline plan generation
- Native support for exogenous events (the world can change independently of the agent's actions)

Interpreter used: indigolog_plain.pl

What we'll cover:

- Action theory: fluents, poss/2, successor state axioms
- Reasoning tasks: Legality, Projection, Regression
- Simple & Reactive controllers, with exogenous events

Indigolo fluent and initial state

```
%% Fluents
prim_fluent(at(_)).           % agent's current room
prim_fluent(has(_)).          % agent carries Item
prim_fluent(item_at(_,_)).    % item_at(Item,Room)
prim_fluent(unlocked(_)).     % unlocked(Lock)
prim_fluent(known_code(_)).   % known_code(Lock)
prim_fluent(blocked(_,_)).    % door temporarily jammed
prim_fluent(last_room(_)).    % anti-cycle guard

%% Successor state axioms (causes_val/4)
causes_val(move(_R1,R2,_L), at(R2), true,  true).
causes_val(move(R1,_R2,_L), at(R1), false, true).

causes_val(pick_up(I,_R), has(I),       true,  true).
causes_val(pick_up(I,R),  item_at(I,R),  false, true).

causes_val(combine(_I1,_I2,I3), has(I3), true,  true).
causes_val(combine(I1,_I2,_I3), has(I1), false, true).
causes_val(combine(_I1,I2,_I3), has(I2), false, true).

causes_val(unlock_with_key(L,_I), unlocked(L), true, true).
causes_val(unlock_with_code(L),   unlocked(L), true, true).
```

Indigoloexogenous action

```
%% Exogenous actions (environment-driven, not chosen by the agent)
exog_action(hint_revealed(_)).       % hint_revealed(Lock) -- code learned early
exog_action(door_jams(_,_)).         % door_jams(R1,R2)    -- door becomes blocked

poss(hint_revealed(_), true).
poss(door_jams(_,_), true).

%% Effects, via the same causes_val/4 mechanism
causes_val(hint_revealed(L),   known_code(L), true, true).
causes_val(door_jams(R1,R2),   blocked(R1,R2), true, true).
causes_val(door_jams(R1,R2),   blocked(R2,R1), true, true).  % symmetric

%% Precondition on move: blocked doors can't be used
poss(move(R1,R2,L), and(at(R1),
                  and(connected(R1,R2,L),
                  and(unlocked(L),
                     neg(blocked(R1,R2))))))).
```

legality task

As first reasoning task we implemented the legality task, which verifies whether a given sequence of actions is legal from s0, executing it for real via indigolog/1.

```
demo_legality :-
  Illegal = [unlock_with_key(l0,key_start), move(r1,r2,l0)],
  ( indigolog(Illegal)
  -> format("UNEXPECTED: illegal sequence was accepted!~n")
  ;  format("As expected, ~w is ILLEGAL from s0 (key not picked up yet).~n", [Illegal])
  ),
  initialize,
  Legal = [pick_up(key_start,r1), unlock_with_key(l0,key_start), move(r1,r2,l0),
           ... , move(r5,r8,l_w4)],
  ( indigolog(Legal)
  -> format("As expected, the full 17-action west-branch plan IS legal from s0.~n")
  ;  format("UNEXPECTED: legal sequence was rejected!~n")
  ).
```

Legal example:

SEQ = [pick_up(key_start,r1), unlock_with_key(l0,key_start), move(r1,r2,l0), ..., move(r5,r8,l_w4)]

pick_up(key_start,r1)
unlock_with_key(l0,key_start)
move(r1,r2,l0)
move(r5,r8,l_w4)
17 actions.
As expected, the full 17-action west-branch plan IS legal from s0.

Illegal example:

SEQ = [unlock_with_key(l0,key_start), move(r1,r2,l0)]

As expected, [unlock_with_key(l0,key_start),move(r1,r2,l0)] is ILLEGAL from s0 (key not picked up yet).

projection tasl

As second reasoning task we implemented the projection task, which checks whether a given fluent holds after executing a sequence of actions. holds/2 is a pure query — it does not execute anything for real, and takes the sequence in reversed order (most recent action first, mirroring do(a,s) nesting).

```
demo_projection :-
  Forward = [pick_up(key_start,r1), unlock_with_key(l0,key_start), move(r1,r2,l0),
             pick_up(key_w1,r2), unlock_with_key(l_w1,key_w1), move(r2,r3,l_w1),
             pick_up(item_p,r3)],
  reverse(Forward, Reversed),
  ( holds(has(item_p), Reversed)
  -> format("Projection OK: has(item_p) holds after ~w.~n", [Forward])
  ;  format("Projection FAILED: has(item_p) does not hold after ~w.~n", [Forward])
  ),
  ( holds(at(r8), Reversed)
  -> format("UNEXPECTED: at(r8) holds already?!~n")
  ;  format("As expected, the agent has NOT reached the exit yet at this point.~n")
  ).
```

Legal example — has(item_p) holds:
COND = has(item_p), SEQ (forward) = [pick_up(key_start,r1), ..., pick_up(item_p,r3)]
Projection OK: has(item_p) holds after [pick_up(key_start,r1),...,pick_up(item_p,r3)].
Negative example — at(r8) does not hold yet:
COND = at(r8), same SEQ
As expected, the agent has NOT reached the exit yet at this point.

regretion task

Slide 23 — Regression Task
As third reasoning task we implemented the regression task, which checks
whether a goal situation is reachable from the initial situation, and
executes the discovered plan if one exists.

```
demo_regression :-
  Goal = some(r, and(exit_room(r), at(r))),   % i.e. goal_reached itself
  ( indigolog(search(while(neg(Goal), any_action)))
  -> format("Goal situation ~w IS reachable from the initial situation.~n", [Goal])
  ;  format("Goal situation ~w is NOT reachable from the initial situation.~n", [Goal])
  ).
```

Legal example — goal reachable:

```
 Goal = some(r, and(exit_room(r), at(r)))
```

```
pick_up(key_start,r1)
unlock_with_key(l0,key_start)
move(r1,r2,l0)
...
move(r5,r8,l_w4)
```

19 actions.
Goal situation some(r,and(exit_room(r),at(r))) IS reachable from the initial situation.
Illegal / unreachable example — not executed live:
An unreachable goal would need while(neg(Goal), any_action) to search
forever, since last_room/1 only blocks an immediate bounce-back, not
longer cycles — no guaranteed termination on this domain. Documented
rather than risked live, the same call made by the Sokoban reference
project for the identical reason.
I

controller simple task

```
proc(do_move,
  pi(r1, pi(r2, pi(l,
    [ ?(room(r1)), ?(room(r2)), ?(or(key_lock(l), code_lock(l))),
      ?(neg(last_room(r2))), move(r1,r2,l) ]
  )))
).
%% ... do_pick_up, do_combine, do_read_clue, do_unlock_key, do_unlock_code
%% similarly defined, each a grounded action template + guard.

proc(any_action,
  ndet(do_move,
  ndet(do_pick_up,
  ndet(do_combine,
  ndet(do_read_clue,
  ndet(do_unlock_key,
      do_unlock_code
  )))))
).

proc(escape_task, while(neg(goal_reached), any_action)).

proc(control_simple, search(escape_task)).
Results — all three instances:
Instance   Result                              Actions
Easy       optimal plan found                      6
Medium     optimal plan found (matches PDDL exactly)  14
Hard       west branch found (2 harmless extra pick-ups)  19
less detour." ~30 secondi.
```

controller reactive task

```
proc(control_reactive, prioritized_interrupts([
  interrupt(neg(goal_reached), search(escape_task))
])).
```

search(escape_task) re-plans the full remaining task on every interrupt cycle, not just one action at a time — necessary because indigolog_plain.pl has no gexec/unset (the constructs sokoban.pl/taxi.pl use to abort and restart a search mid-execution).

Results — all three instances:

Instance	Result	Actions
Easy	optimal plan found	6 (+2 interrupt overhead)
Medium	optimal plan found	14 (+2 interrupt overhead)
Hard	west branch found (2 harmless extra pick-ups)	19 (+2 interrupt overhead)

exogenous event

Both events fire before the first real action — indigolog_plain.pl check

Two mechanisms confirmed working:

- hint_revealed(l_e2) — unlock_with_code(l_e2) fires directly,
- read_clue(l_e2,r6) never called.
- door_jams(r2,r3) — the west branch is never explored at all;
- the plan goes straight through the east branch.

Per

Both events fire before the first real action — indigolog_plain.pl checks exog_occurs/1 before every attempted transition, so queued events are fully incorporated before search(escape_task) is ever invoked.

```
=== The Locked Archive -- live exogenous-event demo ===
Starting Reactive Controller on the HARD instance...
>>> [demo harness] injecting exogenous event: hint_revealed(l_e2)
>>> [demo harness] injecting exogenous event: door_jams(r2,r3)
start_interrupts
pick_up(key_start,r1)
unlock_with_key(l0,key_start)
move(r1,r2,l0)
pick_up(key_w1,r2)
pick_up(key_e1,r2)
unlock_with_key(l_w1,key_w1)
unlock_with_key(l_e1,key_e1)
move(r2,r6,l_e1)
pick_up(item_m,r6)
unlock_with_code(l_e2)
move(r6,r7,l_e2)
pick_up(item_n,r7)
pick_up(key_e3,r7)
combine(item_m,item_n,bonus_item)
unlock_with_key(l_e3,key_e3)
move(r7,r8,l_e3)
stop_interrupts
20 actions.
true
```

conclution

CONCLUSIONS

Same domain, two formalisms, two kinds of insight:

PDDL / Fast Downward

- Admissible search (Blind) always finds the optimum, at a real cost in expanded states
- Inadmissible heuristics (hFF, hadd) are much faster, but can — and did — miss the optimum
- A domain-specific move-cost metric let us make this trade-off concrete and reproducible

IndiGolog

- The same domain supports online execution and exogenous events — neither expressible in PDDL
- Reasoning tasks (Legality, Projection, Regression) let us verify the action theory formally, not just empirically
- Every result shown today is a real execution, not a simulation

SAPIENZA
UNIVERSITÀ DI ROMA

THANK YOU
FOR
THE ATTENTION!